מדריך מתכנת – סימולציית SeeBus

הסימולציה בנויה בצורה מודולרית, כך שניתן לשנות את מודל הביקוש, מדיניות התזמון או פרמטרי הניסוי בלי לשנות את כל המערכת.
המבנה הכללי מפריד בין יצירת העולם, הרצת התחבורה, המדיניות, הניסויים ו-הדיווח.

מבנה הפרויקט

```
seebus_sim/
├── demand/          # יצירת ביקוש ולמידת היסטוריה
├── transit/         # מנוע התחבורה והסימולציה
├── experiments/     # Batch, sweeps וניסויי פרמטרים
├── reporting/       # CSV, גרפים ודוחות HTML
├── config.py        # הגדרות כלליות
└── scenarios.py     # הגדרת מבנה הרשת

run.py               # הרצה בודדת
run_batch.py         # מספר עולמות והשוואת מדיניות
run_suite.py         # חבילת ניסויי פרמטרים
run_tuning.py        # כוונון Historical ו-Hybrid
```

מודל העולם והביקוש

כל עולם סימולציה נוצר באופן עצמאי וכולל ביקוש סינתטי לכל תחנה.
הביקוש מושפע מ:

- סוג התחנה, לדוגמה מגורים, משרדים או מסחר
- שעות עומס בבוקר ובערב
- יום בשבוע
- שונות אקראית בין ימים
- הגעת נוסעים לפי תהליך Poisson
- אירועי ביקוש חריגים וזמניים

הפרמטרים האמיתיים של העולם נחשבים מידע נסתר ואינם נחשפים למדיניות התזמון.
לפני שלב הבדיקה נוצרים ימי אימון, מהם נלמד מודל היסטורי פשוט של קצב הגעת נוסעים לכל תחנה וחלון זמן.

כך נשמרת הפרדה בין:

```
Hidden World
    ↓
Training Data
    ↓
Learned Historical Model
    ↓
Scheduling Policy
```

מודל התחבורה

התרחיש הראשי כולל Hub משותף ומספר קווים קבועים.
כל אוטובוס מתחיל ב-Hub. לאחר Dispatch הוא נוסע:

```
Hub -> Station 1 -> Station 2 -> ... -> Hub
```

האוטובוס אינו זמין לשליחה למסולול חדש עד לסיום כל המסלול.
זמן הנסיעה הוא חלק מהסימולציה ולכן שליחת אוטובוס אינה משרתת את התחנות באופן מיידי.
הסימולציה משתמשת ב-SimPy לניהול הזמן והאירועים.

PolicyContext

קבוצת המדיניות אינן ניגשות ישירות למצב הפנימי של העולם.
במקום זאת הן מקבלות PolicyContext, המכיל רק מידע שמותר להשתמש בו, לדוגמה:

- השעה ויום השבוע
- קווים ותחנות
- מספר אוטובוסים זמינים
- זמן מאז השירות האחרון
- תחזית היסטורית, אם המדיניות משתמשת בה
- נתוני חיישן, אם המדיניות משתמשת בהם

המדיניות אינן מקבלות:

- את קצב הביקוש האמיתי
- את האירועים הנסתרים
- את אורך התור האמיתי ללא מעבר דרך מודל החיישן

עיקרון זה מונע מהאלגוריתם "לרמות" בסימולציה.

מדיניות התזמון

Static

מדיניות קבועה שאינה משתמשת בביקוש.
הקווים מקבלים Dispatch בסדר Round-Robin לפי Headway קבוע, לדוגמה:

```
Route 1 -> Route 2 -> ... -> Route 10 -> Route 1
```

Historical

משתמשת בביקוש שנלמד מימי האימון.
המדיניות מעריכה כמה נוסעים צפויים להצטבר ומבצעת Dispatch כאשר הביקוש החזוי עובר סף מסוים.
ברירת המחדל היא:

```
historical threshold = 35% of bus capacity
```

Sensor

משתמשת במדידות תפוסה בזמן אמת.
לכל קו מחושב ממוצע של אותות החיישנים בתחנות הקו.
ברירת המחדל:

```
sensor target occupancy = 20%
```

Hybrid

משלבת את שני מקורות המידע לאחר נרמול:

```
Hybrid pressure =
history_weight × historical_pressure
+
sensor_weight × sensor_pressure
```

ברירת המחדל:

```
40% Historical
60% Sensor
```

Headway safeguards

למדיניות האדפטיביות יש שני גבולות:

```
minimum headway
maximum headway
```

minimum headway מונע שליחת מספר אוטובוסים ברצף לאותו קו.
maximum headway מונע starvation: קו שלא קיבל שירות זמן רב הופך לזכאי ל-Dispatch גם כאשר הביקוש שלו נמוך.

בתרחיש הראשי:

```
Nominal headway = 15 min
Minimum headway = 7.5 min
Maximum headway = 22.5 min
```

מודל החיישן

החיישן אינו מחזיר את התפוסה האמיתית בצורה מושלמת.
התהליך הוא:

```
True occupancy
      ↓
Gaussian estimation error
      ↓
Clamp to 0–100%
      ↓
1% reporting resolution
      ↓
Server
```

ניתן גם לדמות Packet Dropout.
כאשר הודעה אובדת, השרת ממשיך להשתמש במדידה האחרונה שהתקבלה בהצלחה עד להגעת הודעה חדשה.

ניסויים

run_batch.py מריץ מספר עולמות ומשווה בין המדיניות על אותם תנאים.
run_suite.py מריץ Sweeps של פרמטרים כגון:

-
- מספר אוטובוסים
- תדירות אירועים
- שגיאת חיישן
- Packet Dropout
- משקל Sensor/History ב-Hybrid

`run_tuning.py` מבצע Sensitivity Analysis על:

- Historical threshold
- Hybrid sensor weight

כל המדיניות בתוך אותו עולם מקבלות את אותם נוסעים ואותם אירועים.

מדדים

המדדים המרכזיים הם:

- Mean passenger waiting time
- P95 waiting time
- Maximum queue
- Service rate
- Fleet utilization
- Passenger-capacity utilization

בנוסף מתבצעת בדיקת שימור נוסעים:

```
Generated =
Completed + Onboard + Waiting
```

הוספת Policy חדש

כדי להוסיף מדיניות חדשה יש לבצע את השלבים הבאים:

1. ליצור מחלקת Policy חדשה
2. לממש את ממשק ההחלטה הקיים
3. להשתמש רק במידע מתוך PolicyContext
4. להחזיר:
$DispatchDecision(route_id = \dots)$
או
$None$
5. להוסיף את המדיניות ל-Policy Specs של הניסויים

בדרך כלל אין צורך לשנות את מנוע התחבורה או מערכת הדיווח.

הוספת Scenario חדש

תרחישים מוגדרים ב-scenarios.py
Scenario קובע בין היתר:

- מספר קווים
- מספר תחנות בכל קו
- מספר אוטובוסים
- קיבולת אוטובוס
- זמן נסיעה בין תחנות
- Headw - בסיסי

מומלץ לשמור את מבנה הרשת בנפרד מהפרמטרים הניסויים, כדי שניתן יהיה לשנות כל אחד מהם באופן עצמאי.

בדיקות

לפני שינוי משמעותי יש להריץ:

```
python -m pytest
```

הבדיקות מכסות בין היתר:

- שימור נוסעים
- Seeds דטרמיניסטיים
- השוואה בין Full ו-Compact logging
- Sparse arrivals
- התנהגות Sensor ו-Hybrid
- Fallback של Hybrid כאשר אין נתוני חיישן
- תקינות Sweeps ו-Checkpoints

עקרונות חשובים לפיתוח

מומלץ לשמור על ההפרדה הבאה:

```
World generation
      ↓
Transit simulation
      ↓
Scheduling policies
      ↓
Experiment orchestration
      ↓
Reporting
```

מדיניות תזמון לא צריכה לדעת כיצד נוצר העולם, ומנוע הסימולציה לא צריך להכיר את מטרת הניסוי.
הפרדה זו מאפשרת להוסיף אלגוריתמים וניסויים חדשים תוך שמירה על השוואה הוגנת ועל קוד שקל לבדוק ולתחזק.